Restaurant Menu System

PHP MVC (Core PHP + MySQL) - Code with Explanation

This system manages restaurant menu items. Users can **add**, **view**, **edit** and **delete** items (CRUD). It uses Core PHP, MySQL and MVC architecture, and is designed for XAMPP and VS Code.

Request flow: Browser -> index.php -> Controller -> Model (database) -> Controller -> View -> Browser

Part	Role	File
Model	Database queries only	models/MenuItem.php
View	HTML only (what the user sees)	views/*.php
Controller	Receives requests, connects Model and View	controllers/MenuController.php

1. Folder Structure

Create a folder named **restaurant_mvc** inside C:\xampp\htdocs.

```
restaurant_mvc/
|-- index.php
|-- config/
|   `-- database.php
|-- models/
|   `-- MenuItem.php
|-- controllers/
|   `-- MenuController.php
`-- views/
    |-- menu_list.php
    |-- menu_add.php
    `-- menu_edit.php
```

Explanation: index.php is the single entry point. config/ holds the connection, models/ the SQL, controllers/ the request logic, and views/ the HTML pages. Files must be inside htdocs because that is Apache's web root.

2. Database

Open http://localhost/phpmyadmin and run:

```sql
CREATE DATABASE restaurant_db;

USE restaurant_db;

CREATE TABLE menu_items (
    id INT AUTO_INCREMENT PRIMARY KEY,
    item_code VARCHAR(50),
    item_name VARCHAR(100),
    category VARCHAR(100),
    status VARCHAR(50),
    price DECIMAL(10,2)
);
```

- **id** is a unique number created automatically; it identifies which record to edit or delete.
- **VARCHAR** stores text. item_code is text because it may contain letters and dashes (e.g. MN-001).
- **DECIMAL(10,2)** stores prices exactly (e.g. 149.50) without the rounding errors of FLOAT.
- If restaurant_db already exists, skip the CREATE DATABASE line.

3. Database Connection

File: config/database.php

```php
<?php

$conn = new mysqli("localhost", "root", "", "restaurant_db");

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

?>
```

- **mysqli(host, username, password, database)** connects PHP to MySQL.
- XAMPP default: user root, empty password.
- If the connection fails (e.g. MySQL not running), die() stops the script and shows the error.
- $conn is reused by the rest of the project.

4. Model

File: models/MenuItem.php

```php
<?php

class MenuItem
{
    private $conn;

    public function __construct($db)
    {
        $this->conn = $db;
    }

    // READ all
    public function getAll()
    {
        return $this->conn->query(
            "SELECT * FROM menu_items ORDER BY category, item_name"
        );
    }

    // READ one
    public function getById($id)
    {
        $stmt = $this->conn->prepare("SELECT * FROM menu_items WHERE id = ?");
        $stmt->bind_param("i", $id);
        $stmt->execute();

        return $stmt->get_result()->fetch_assoc();
    }

    // CREATE
    public function add($code, $name, $category, $status, $price)
    {
        $stmt = $this->conn->prepare(
            "INSERT INTO menu_items (item_code, item_name, category, status, price)
             VALUES (?, ?, ?, ?, ?)"
        );

        $stmt->bind_param("ssssd", $code, $name, $category, $status, $price);

        return $stmt->execute();
    }
```

```
    // UPDATE
    public function update($id, $code, $name, $category, $status, $price)
    {
        $stmt = $this->conn->prepare(
            "UPDATE menu_items
             SET item_code = ?, item_name = ?, category = ?, status = ?, price = ?
             WHERE id = ?"
        );

        $stmt->bind_param("ssssdi", $code, $name, $category, $status, $price, $id);

        return $stmt->execute();
    }

    // DELETE
    public function delete($id)
    {
        $stmt = $this->conn->prepare("DELETE FROM menu_items WHERE id = ?");
        $stmt->bind_param("i", $id);

        return $stmt->execute();
    }
}

?>
```

- The class holds **all the SQL**. The constructor receives the connection and stores it in $this->conn.
- **getAll()** returns every row, sorted by category then name. **getById()** returns one row as an array, used to pre-fill the edit form.
- **add(), update(), delete()** use **prepared statements**: the ? placeholders prevent SQL injection.
- **bind_param types:** s = string, d = decimal, i = integer. "ssssd" = four strings and one decimal. In update(), "ssssdi" has an extra i for the id in WHERE.
- The variable order must match the order of the ? marks.

5. Controller

File: controllers/MenuController.php

```
<?php

require_once "models/MenuItem.php";

class MenuController
{
    private $menu;

    public function __construct($db)
    {
        $this->menu = new MenuItem($db);
    }

    // Show list
    public function index()
    {
        $items = $this->menu->getAll();

        include "views/menu_list.php";
    }

    // Show add form / save
    public function add()
    {
```

```
        if (isset($_POST['save'])) {

            $this->menu->add(
                $_POST['item_code'],
                $_POST['item_name'],
                $_POST['category'],
                $_POST['status'],
                $_POST['price']
            );

            header("Location: index.php");
            exit;
        }

        include "views/menu_add.php";
    }

    // Show edit form / update
    public function edit()
    {
        $id = (int) $_GET['id'];

        $item = $this->menu->getById($id);

        if (isset($_POST['update'])) {

            $this->menu->update(
                $id,
                $_POST['item_code'],
                $_POST['item_name'],
                $_POST['category'],
                $_POST['status'],
                $_POST['price']
            );

            header("Location: index.php");
            exit;
        }

        include "views/menu_edit.php";
    }

    // Delete
    public function delete()
    {
        $id = (int) $_GET['id'];

        $this->menu->delete($id);

        header("Location: index.php");
        exit;
    }
}

?>
```

- The controller is the "middle man": it receives the request, calls the Model, then loads a View.
- **index()** gets all items, then include loads the list view where $items is available.
- **add()** checks isset($_POST['save']). If the form was submitted it saves and redirects; otherwise it shows the empty form.
- **edit()** reads id from the URL (index.php?action=edit&id=3); (int) forces it to be a number. It loads the item for the form and saves if update was clicked.
- **delete()** removes the item and redirects to the list.
- **header("Location: ...") + exit** redirects after saving, which prevents duplicate records when the page is refreshed.

6. Views

A. Menu List

File: views/menu_list.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Restaurant Menu</title>
</head>
<body>

    <h2>Restaurant Menu</h2>

    <a href="index.php?action=add">Add Menu Item</a>
    <br><br>

    <table border="1" cellpadding="10">
        <tr>
            <th>ID</th>
            <th>Item Code</th>
            <th>Item Name</th>
            <th>Category</th>
            <th>Status</th>
            <th>Price</th>
            <th>Action</th>
        </tr>

        <?php while ($row = $items->fetch_assoc()) { ?>
        <tr>
            <td><?php echo $row['id']; ?></td>
            <td><?php echo htmlspecialchars($row['item_code']); ?></td>
            <td><?php echo htmlspecialchars($row['item_name']); ?></td>
            <td><?php echo htmlspecialchars($row['category']); ?></td>
            <td><?php echo htmlspecialchars($row['status']); ?></td>
            <td><?php echo number_format($row['price'], 2); ?></td>
            <td>
                <a href="index.php?action=edit&id=<?php echo $row['id']; ?>">Edit</a>
                |
                <a href="index.php?action=delete&id=<?php echo $row['id']; ?>"
                   onclick="return confirm('Delete this item?')">Delete</a>
            </td>
        </tr>
        <?php } ?>
    </table>

</body>
</html>
```

- **while ($row = $items->fetch_assoc())** repeats once per row and builds one table row each time.
- **htmlspecialchars()** makes special characters safe to display (prevents XSS).
- **number_format($row['price'], 2)** shows the price with two decimals.
- The links carry action and id so the router and controller know what to do.
- **onclick="return confirm(...)"** asks before deleting; Cancel returns false and the link is not followed.

B. Add Form

File: views/menu_add.php

```
<!DOCTYPE html>
<html>
<head>
```

```
    <title>Add Menu Item</title>
</head>
<body>

    <h2>Add Menu Item</h2>

    <form method="POST">

        Item Code:
        <input type="text" name="item_code" required>
        <br><br>

        Item Name:
        <input type="text" name="item_name" required>
        <br><br>

        Category:
        <select name="category" required>
            <option value="">Select Category</option>
            <option value="Appetizer">Appetizer</option>
            <option value="Main Course">Main Course</option>
            <option value="Dessert">Dessert</option>
            <option value="Drinks">Drinks</option>
        </select>
        <br><br>

        Status:
        <select name="status" required>
            <option value="Available">Available</option>
            <option value="Sold Out">Sold Out</option>
        </select>
        <br><br>

        Price:
        <input type="number" name="price" step="0.01" min="0" required>
        <br><br>

        <button type="submit" name="save">Save</button>

    </form>

    <br>
    <a href="index.php">Back</a>

</body>
</html>
```

- With no action attribute, the form submits to the same URL (index.php?action=add), where the controller handles it.
- Each field's **name** becomes a key in $_POST (e.g. $_POST['item_name']).
- The button's name="save" is what the controller checks with isset($_POST['save']).
- **required**, **step="0.01"** and **min="0"** give basic browser validation.

C. Edit Form

File: views/menu_edit.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Edit Menu Item</title>
</head>
<body>

    <h2>Edit Menu Item</h2>

    <form method="POST">
```

```
        Item Code:
        <input type="text" name="item_code"
        value="<?php echo htmlspecialchars($item['item_code']); ?>" required>
        <br><br>

        Item Name:
        <input type="text" name="item_name"
        value="<?php echo htmlspecialchars($item['item_name']); ?>" required>
        <br><br>

        Category:
        <select name="category" required>
            <option value="Appetizer" <?php if ($item['category'] == "Appetizer") echo "selected"; ?>>Appetizer</option>
            <option value="Main Course" <?php if ($item['category'] == "Main Course") echo "selected"; ?>>Main Course</o
            <option value="Dessert" <?php if ($item['category'] == "Dessert") echo "selected"; ?>>Dessert</option>
            <option value="Drinks" <?php if ($item['category'] == "Drinks") echo "selected"; ?>>Drinks</option>
        </select>
        <br><br>

        Status:
        <select name="status" required>
            <option value="Available" <?php if ($item['status'] == "Available") echo "selected"; ?>>Available</option>
            <option value="Sold Out" <?php if ($item['status'] == "Sold Out") echo "selected"; ?>>Sold Out</option>
        </select>
        <br><br>

        Price:
        <input type="number" name="price" step="0.01" min="0"
        value="<?php echo htmlspecialchars($item['price']); ?>" required>
        <br><br>

        <button type="submit" name="update">Update</button>

    </form>

    <br>
    <a href="index.php">Back</a>

</body>
</html>
```

- Same as the add form, but fields are **pre-filled** from the $item array loaded by the controller.
- echo "selected" marks the current category and status in the dropdowns.
- The button is named update, which the controller checks with isset($_POST['update']).

7. Main File (Router)

File: index.php

```php
<?php

require_once "config/database.php";
require_once "controllers/MenuController.php";

$controller = new MenuController($conn);

$action = isset($_GET['action']) ? $_GET['action'] : 'index';

if ($action == 'add') {
    $controller->add();
} elseif ($action == 'edit') {
    $controller->edit();
} elseif ($action == 'delete') {
    $controller->delete();
```

```php
} else {
    $controller->index();
}

?>
```

- It loads the connection and controller, then creates the controller with $conn.
- It reads ?action= from the URL and defaults to "index" (the list page).
- The if / elseif chain is the **router**:

URL	Method called	Result
index.php	index()	Menu list
index.php?action=add	add()	Add form / save
index.php?action=edit&id=2	edit()	Edit form / update
index.php?action=delete&id=2	delete()	Delete and redirect

8. Running the Program

1. Start **Apache** and **MySQL** in the XAMPP Control Panel.
2. Create the restaurant_db database and menu_items table in phpMyAdmin.
3. Create the files exactly as in the folder structure (the require_once paths depend on it).
4. Save the project in C:\xampp\htdocs\restaurant_mvc.
5. Open **http://localhost/restaurant_mvc/** in your browser.

Optional sample data:

```sql
INSERT INTO menu_items (item_code, item_name, category, status, price) VALUES
('MN-001', 'Chicken Adobo', 'Main Course', 'Available', 149.00),
('MN-002', 'Halo-Halo', 'Dessert', 'Available', 99.50),
('MN-003', 'Iced Tea', 'Drinks', 'Sold Out', 45.00);
```

CRUD Summary

Operation	Model	Controller	View
Create	add()	add()	menu_add.php
Read	getAll(), getById()	index(), edit()	menu_list.php
Update	update()	edit()	menu_edit.php
Delete	delete()	delete()	(redirects to list)

Key Points for the Exam

- **MVC** separates SQL (Model), logic (Controller) and HTML (View).
- **Prepared statements** (prepare, bind_param) prevent SQL injection.
- **htmlspecialchars()** prevents XSS.
- **Post/Redirect/Get** (header + exit) prevents duplicate submissions.
- **DECIMAL** is used for prices to avoid rounding errors.
- **Possible improvement:** deleting through a GET link is not ideal in real projects; a POST form is safer.